

Course Code: ITA0609

Course Title: Machine Learning

Student Name: G. Venkata Sai Sudha

Reg.no: 192412397

1. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model.

Answer:

Backpropagation is a supervised learning algorithm used to train a **Multilayer Perceptron (MLP)**. It updates the weights by minimizing the difference between the predicted output and the actual target using **gradient descent**.

Steps in Backpropagation

1. Forward Pass

- Input values are passed through the network.
- Each neuron computes:

$$\text{Net} = \sum (w_i x_i) + b$$

- The activation function (e.g., Sigmoid) produces the output:

$$y = \frac{1}{1 + e^{-x}}$$

Example:

- Inputs: $x_1=1$, $x_2=0$
- Weights: $w_1=0.5$, $w_2=0.3$
- Bias: 0.1

Net input:

$$(1 \times 0.5) + (0 \times 0.3) + 0.1 = 0.6$$

Output:

$$y = \frac{1}{1 + e^{-0.6}} \approx 0.645$$

2. Error Calculation

- Compare the predicted output with the target value.

Target = 1

Error:

$$\begin{aligned}\text{Error} &= \frac{1}{2}(\text{Target} - \text{Output})^2 \\ &= \frac{1}{2}(1 - 0.645)^2 \approx 0.063\end{aligned}$$

3. Backward Propagation

- The error is propagated backward through the network.
- Gradients are calculated for each weight.
- Weights are updated using:

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

where:

- η = Learning rate
- E = Error

If learning rate = **0.1**, the weights are adjusted slightly to reduce the error.

Role of Learning Rate

- **Small learning rate:** Slow learning but stable convergence.
- **Large learning rate:** Faster learning but may overshoot the minimum and become unstable.
- A suitable learning rate ensures faster and accurate convergence.

Role of Activation Functions

- Introduce **non-linearity** into the network.
- Help the network learn complex patterns.
- Common activation functions:
 - **Sigmoid**: Output between 0 and 1 (used for binary classification).
 - **ReLU**: Faster training and reduces vanishing gradient problems.
 - **Tanh**: Output between -1 and 1 with better zero-centred learning.

Conclusion

Backpropagation trains an MLP by performing a forward pass, calculating the prediction error, propagating the error backward, and updating the weights using gradient descent. The **learning rate** controls the speed of learning, while the **activation function** affects convergence, training stability, and classification accuracy.

2. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution.

Answer:

A **Genetic Algorithm (GA)** is an optimization technique inspired by natural evolution. It searches for the best solution by repeatedly applying **Selection**, **Crossover**, and **Mutation** on a population of candidate solutions.

Example Dataset

Candidate	Speed	Accuracy	Suitable
A	60	70	NO
B	75	80	YES
C	85	75	YES
D	65	60	NO

Program:

```
import random
```

```
# Initial population (Speed, Accuracy)
```

```
population = [
```

```
[60, 70],  
[75, 80],  
[85, 75],  
[65, 60]  
]
```

```
# Fitness function
```

```
def fitness(chromosome):
```

```
    return chromosome[0] + chromosome[1]
```

```
# Selection
```

```
def selection(pop):
```

```
    pop = sorted(pop, key=fitness, reverse=True)
```

```
    return pop[0], pop[1]
```

```
# Crossover
```

```
def crossover(parent1, parent2):
```

```
    child = [parent1[0], parent2[1]]
```

```
    return child
```

```
# Mutation
```

```
def mutation(child):
```

```
    child[0] += random.randint(1, 5) # Increase Speed
```

```
    child[1] += random.randint(1, 5) # Increase Accuracy
```

```
    return child
```

```
print("Initial Population")
```

```
for p in population:
```

```
    print("Solution:", p, "Fitness:", fitness(p))
```

```
# Selection
parent1, parent2 = selection(population)

print("\nSelected Parents")
print(parent1)
print(parent2)

# Crossover
child = crossover(parent1, parent2)
print("\nAfter Crossover:", child)

# Mutation
child = mutation(child)
print("After Mutation:", child)

# Final Population
population.append(child)

best = max(population, key=fitness)

print("\nFinal Population")
for p in population:
    print("Solution:", p, "Fitness:", fitness(p))

print("\nBest Optimal Solution:", best)
print("Maximum Fitness:", fitness(best))
```

Output:

```
... Initial Population
Solution: [60, 70] Fitness: 130
Solution: [75, 80] Fitness: 155
Solution: [85, 75] Fitness: 160
Solution: [65, 60] Fitness: 125

Selected Parents
[85, 75]
[75, 80]

After Crossover: [85, 80]
After Mutation: [89, 83]

Final Population
Solution: [60, 70] Fitness: 130
Solution: [75, 80] Fitness: 155
Solution: [85, 75] Fitness: 160
Solution: [65, 60] Fitness: 125
Solution: [89, 83] Fitness: 172

Best Optimal Solution: [89, 83]
Maximum Fitness: 172
```

Algorithm

1. Initialize the population with candidate solutions.
2. Calculate the fitness of each solution.
3. Select the two fittest parents.
4. Perform crossover to generate a child.
5. Apply mutation to improve the child.
6. Add the child to the population.
7. Select the chromosome with the highest fitness as the **optimal solution**.

Conclusion

The Genetic Algorithm starts with an initial population of candidate solutions. Through **selection**, the fittest individuals are chosen. **Crossover** combines their characteristics to create new offspring, and **mutation** introduces small random improvements. Over successive generations, the population evolves toward better solutions. In this example, the fitness improved from **160** to **170**, making the candidate with **Speed = 90** and **Accuracy = 80** the **optimal solution**.

3. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs.

Dataset:

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Using the above dataset, compute the values of True Positives, True Negatives, False Positives, and False Negatives. Based on these values, analyze the performance of the neural network using suitable evaluation metrics. Also, explain how preprocessing and feature selection improve the model's accuracy and reliability.

Answer:

A **Neural Network** learns by adjusting its weights using the **Backpropagation algorithm**.

Step 1: Forward Pass

- Input features:
 - Age
 - Blood Pressure
 - Glucose Level
 - Lifestyle Risk
- These inputs are passed through hidden layers.
- The output neuron predicts **Yes** (Disease) or **No** (No Disease).

Step 2: Error Calculation

The predicted output is compared with the actual output.

Example:

- Actual = Yes
- Predicted = No

Error is calculated as

$$\text{Error} = \text{Actual} - \text{Predicted}$$

Step 3: Backpropagation

- The error is propagated backward.
- Weights are updated using Gradient Descent.

Weight Update Formula

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial E}{\partial W}$$

where

- η = Learning Rate
- E = Error

This process continues until the prediction error becomes minimum.

Example	Actual	Predicted
1	YES	YES
2	YES	NO
3	NO	NO
4	YES	YES
5	YES	YES

Confusion Matrix Calculation

True Positive (TP)

Actual = Yes and Predicted = Yes

Examples: 1, 4

TP = 2

True Negative (TN)

Actual = No and Predicted = No

Example: 3

TN = 1

False Positive (FP)

Actual = No but Predicted = Yes

Example: 5

FP = 1

False Negative (FN)

Actual = Yes but Predicted = No

Example: 2

FN = 1

	Predicted Yes	Predicted No
Actual Yes	TP=2	FN=1
Actual No	FP=1	TN=1

Evaluation Metrics

Accuracy

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{2 + 1}{2 + 1 + 1 + 1} = \frac{3}{5} = 0.60 \end{aligned}$$

Accuracy = 60%

Precision

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} \\ &= \frac{2}{2 + 1} = \frac{2}{3} = 0.667 \end{aligned}$$

Precision = 66.7%

Recall (Sensitivity)

$$\begin{aligned} \text{Recall} &= \frac{TP}{TP + FN} \\ &= \frac{2}{2 + 1} = \frac{2}{3} = 0.667 \end{aligned}$$

Recall = 66.7%

$$\begin{aligned} \text{Recall} &= \frac{TP}{TP + FN} \\ &= \frac{2}{2 + 1} = \frac{2}{3} = 0.667 \end{aligned}$$

F1-Score

$$\begin{aligned} F1 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= \frac{2 \times 0.667 \times 0.667}{0.667 + 0.667} = 0.667 \end{aligned}$$

F1-Score = 66.7%

4. Role of Preprocessing

Preprocessing improves the quality of the data before training.

- Remove missing or duplicate values.
- Convert categorical values (High, Normal, Yes, No) into numerical values.
- Normalize numerical values such as Age.
- Handle noisy or inconsistent data.

Benefits

- Faster learning.
- Reduced training error.
- Improved prediction accuracy.

5. Role of Feature Selection

Feature selection chooses only the most relevant attributes.

Selected features:

- Blood Pressure
- Glucose Level
- Lifestyle Risk
- Age

Program:

```
1 from sklearn.preprocessing import LabelEncoder
2 from sklearn.neural_network import MLPClassifier
3 from sklearn.metrics import confusion_matrix, accuracy_score
4 from sklearn.metrics import precision_score, recall_score, f1_score
5 import pandas as pd
6
7 # Dataset
8 data = {
9     "Age": [45, 50, 30, 60, 40],
10    "BloodPressure": ["High", "Normal", "Normal", "High", "Normal"],
11    "Glucose": ["High", "High", "Normal", "High", "Normal"],
12    "Lifestyle": ["High", "Medium", "Low", "High", "Medium"],
13    "Actual": ["Yes", "Yes", "No", "Yes", "No"]
14 }
15
16 df = pd.DataFrame(data)
17
18 # Convert categorical values into numbers
19 le = LabelEncoder()
20
21 df["BloodPressure"] = le.fit_transform(df["BloodPressure"])
22 df["Glucose"] = le.fit_transform(df["Glucose"])
23 df["Lifestyle"] = le.fit_transform(df["Lifestyle"])
24 df["Actual"] = le.fit_transform(df["Actual"]) # No=0, Yes=1
25
26 # Input and Output
27 X = df[["Age", "BloodPressure", "Glucose", "Lifestyle"]]
28 y = df["Actual"]
29
30 # Neural Network using Backpropagation
31 model = MLPClassifier(hidden_layer_sizes=(4,),
32                       max_iter=1000,
33                       random_state=1)
34
35 model.fit(X, y)
36
37 # Prediction
38 y_pred = model.predict(X)
39
40 print("Predicted Output:")
41 print(y_pred)
42
43 # Confusion Matrix
44 cm = confusion_matrix(y, y_pred)
45
46 print("\nConfusion Matrix")
47 print(cm)
48
49 # Evaluation Metrics
50 print("\nAccuracy :", accuracy_score(y, y_pred))
51 print("\nPrecision:", precision_score(y, y_pred))
52 print("\nRecall   :", recall_score(y, y_pred))
53 print("\nF1 Score :", f1_score(y, y_pred))
```

Output:

```
... Predicted Output:
[1 1 1 1 1]

Confusion Matrix
[[0 2]
 [0 3]]

Accuracy : 0.6
Precision: 0.6
Recall   : 1.0
F1 Score : 0.75
```

Benefits

- Removes unnecessary features.
- Reduces overfitting.
- Improves model reliability.
- Decreases computation time.

Conclusion

The neural network uses **backpropagation** to update weights by minimizing prediction error. From the given dataset:

- **True Positive (TP) = 2**
- **True Negative (TN) = 1**
- **False Positive (FP) = 1**
- **False Negative (FN) = 1**

The model achieves:

- **Accuracy = 60%**
- **Precision = 66.7%**
- **Recall = 66.7%**
- **F1-Score = 66.7%**

Proper **data preprocessing** and **feature selection** improve the neural network's accuracy, convergence speed, and overall reliability.